

Experiment trackers record results. Agents need live access to improve.

What experiment trackers log

mlflow

- metrics (scalars)
- parameters
- artifacts (files, models)
- runs and metadata

OPTUNA

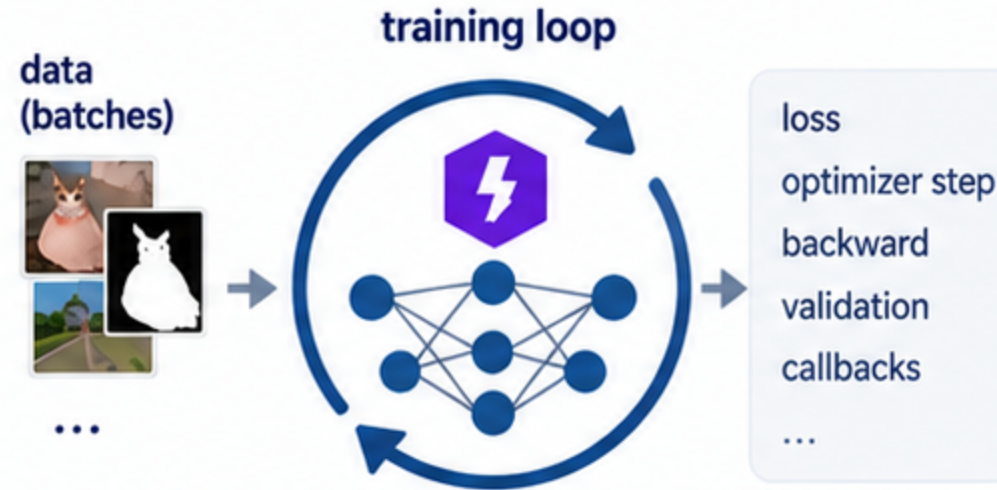
- trial parameters
- objective values
- search history
- best trial

neptune

- metrics and parameters
- artifacts (models, images)
- run comparison
- dashboards

Great for tracking and comparing runs.

A PyTorch Lightning training run



Much of the useful state is only available inside the running process.



What's missing when something goes wrong



no access to internals

tensors, activations, gradients, optimizer state



can't profile the live run

where time and memory are spent



limited debugging context

need to reproduce or add custom logging



no way to intervene

can't pause, inspect, or adjust the run



ephemeral runtime information

detailed traces and intermediate data are not persisted by default

Agents can do more with live access



- ✓ observe the current state
- ✓ experiment with changes
- ✓ close the optimization loop

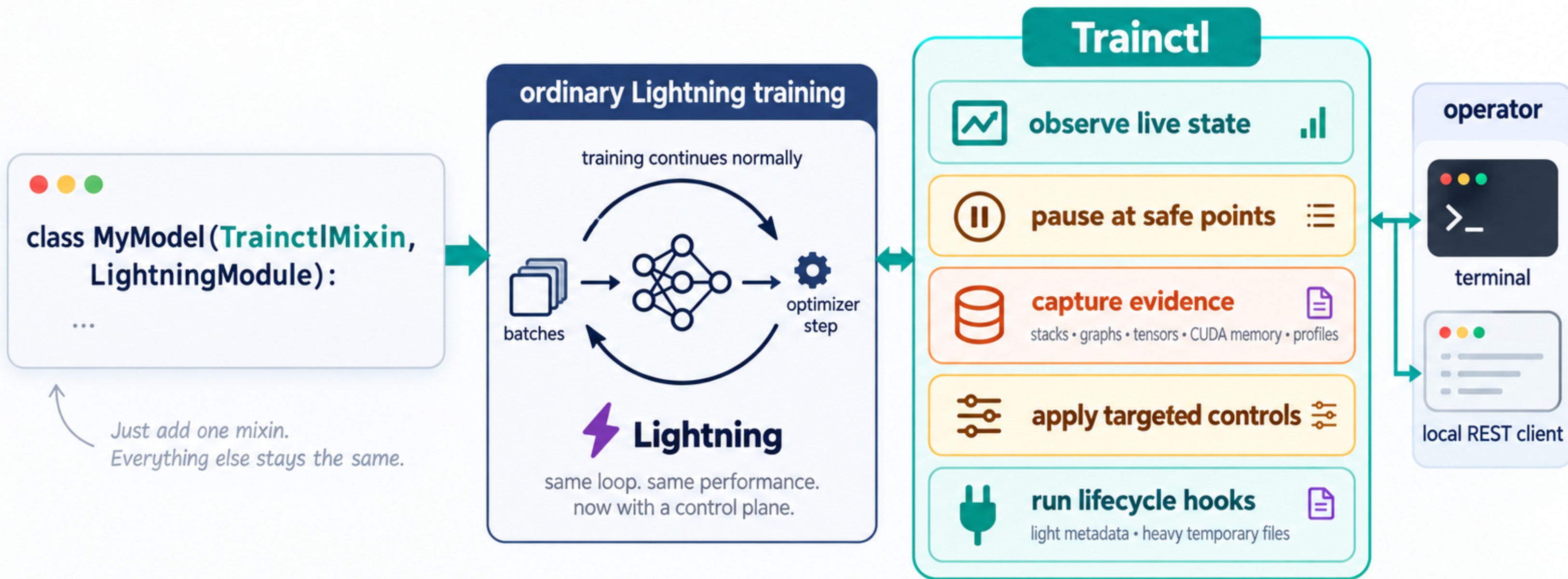
Give agents the right tools and data.



Humans and agents both benefit from live observability and control.

Faster debugging. Better runs.

Add one mixin. Gain a live Lightning control plane.



Inspect, diagnose, and act while the run is alive

One live training run. Three ways Trainctl helps.

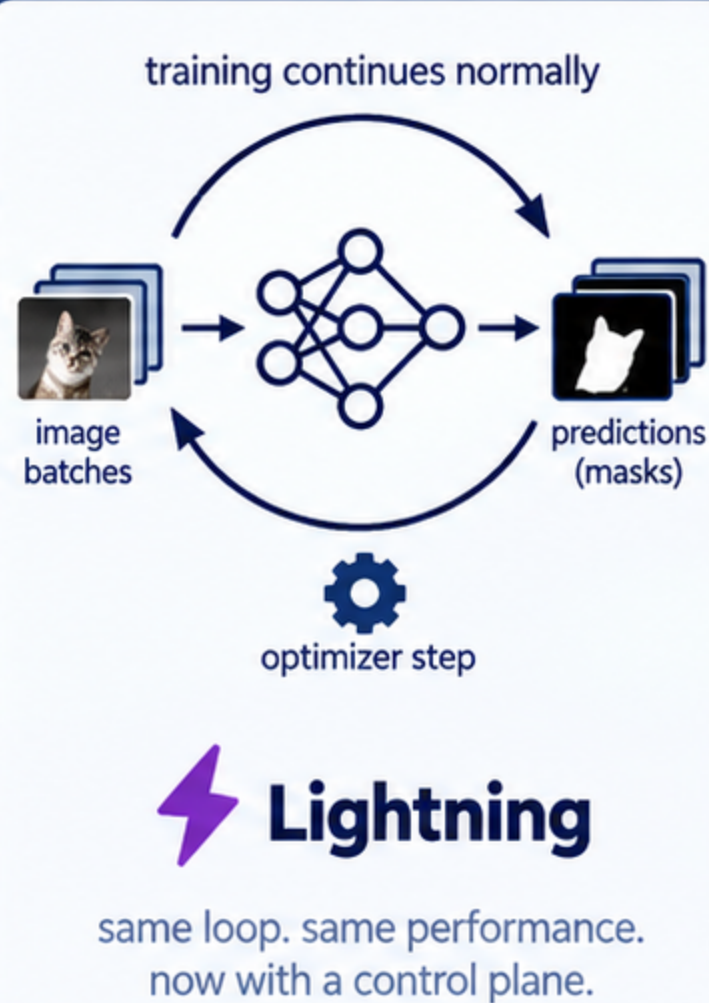
1 debug the model

Inspect what's happening, right now, with real artifacts.



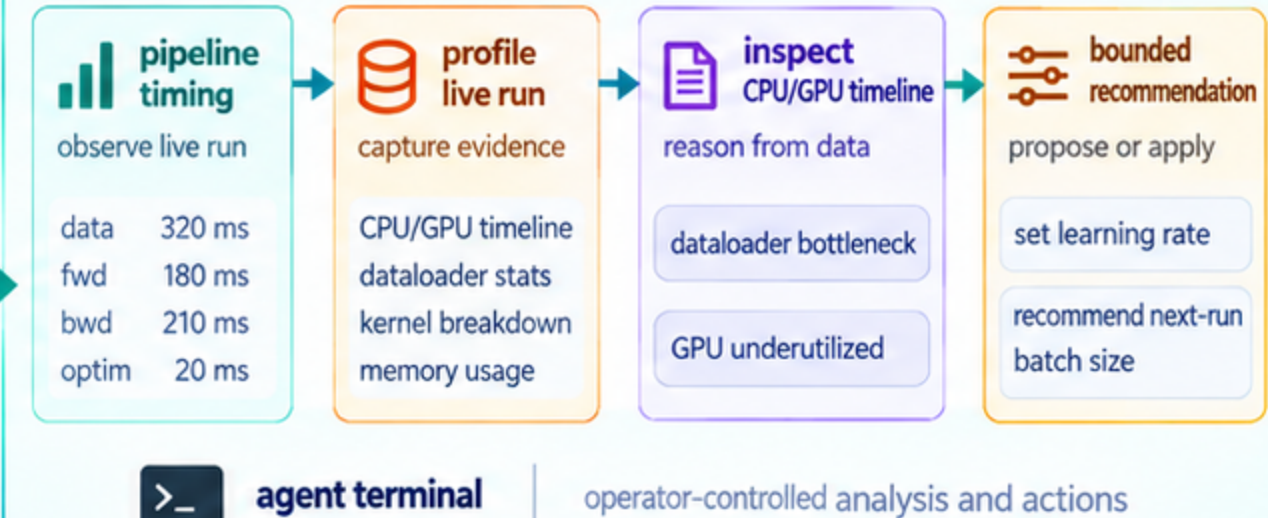
image segmentation training

ordinary Lightning run + TrainctlMixin



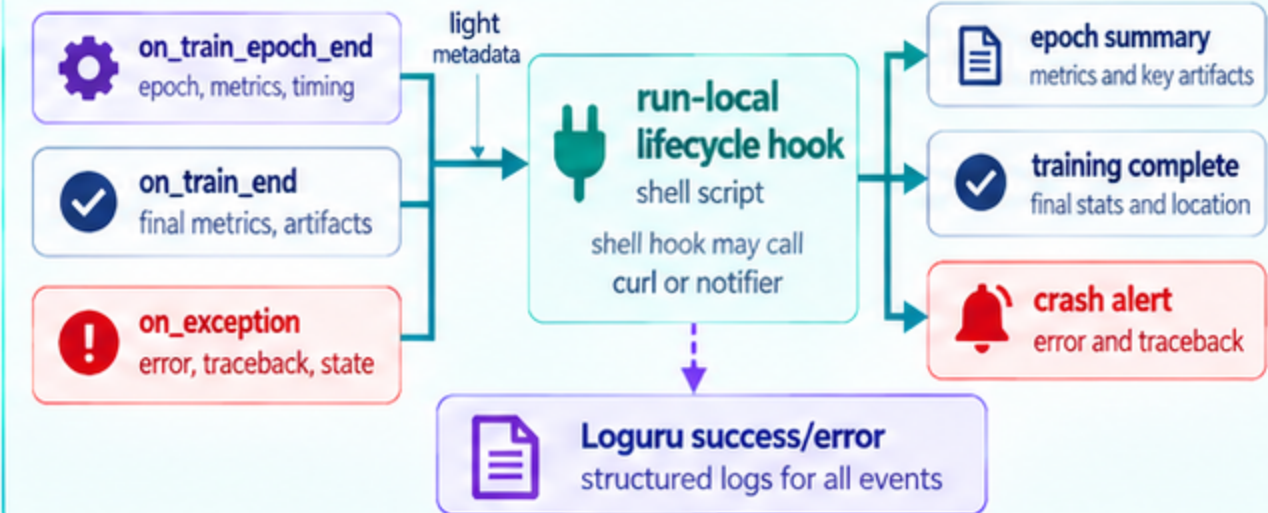
2 let an agent optimize performance

The agent observes, reasons from evidence, and proposes safe actions.



3 automate lifecycle notifications

React to important events with lightweight, local hooks.



Debug what happened. Optimize what is slow. Notify on key events.

The mixin is thin; the live run stays in control

Operator and external tools



REST client / curl
read state, enqueue commands



read-only FUSE mount
browse live artifacts and files



py-spy — read-only attach
profiling without stopping



GDB — high-risk Python injection
can modify process state

Trainctl runtime added by TrainctlMixin



thin Lightning integration seam
TrainctlMixin

composes
into



immutable live state
metrics • model info • optimizer state • etc.



REST + FUSE access surfaces
read state • request captures • download artifacts



typed command queue
validated commands • enqueue only



cooperative hold gate
pause at safe points • deterministic handoff

commands
(after hold)



artifact store
checkpoints • graphs • tensors • profiles

capture operations



lifecycle hook dispatcher
on_train_start • on_batch_end • on_train_end • etc.

callback points



Loguru diagnostics
structured logs • events • errors • tracebacks

diagnostic
records

ordinary Lightning module and Trainer



LightningModule
your model, training_step, validation_step, etc.

training thread
(runs as usual)



batches



optimizer
step

Trainer



training thread
processes batches normally

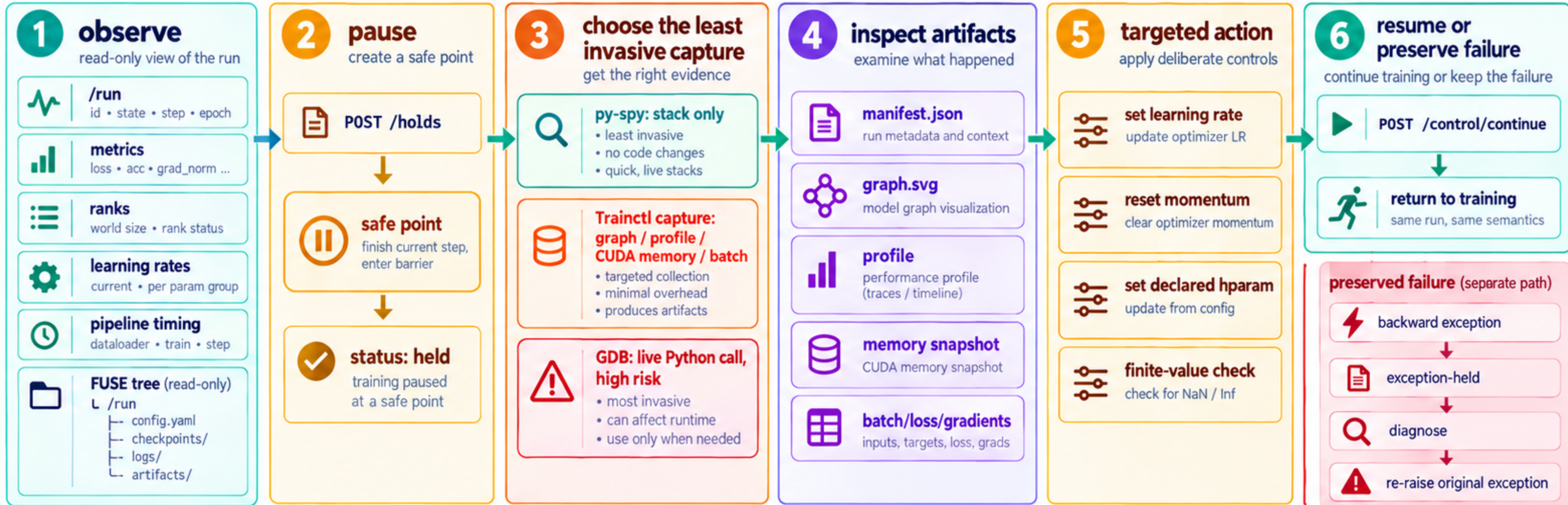


model • batch • gradients • optimizer
ordinary Lightning training loop

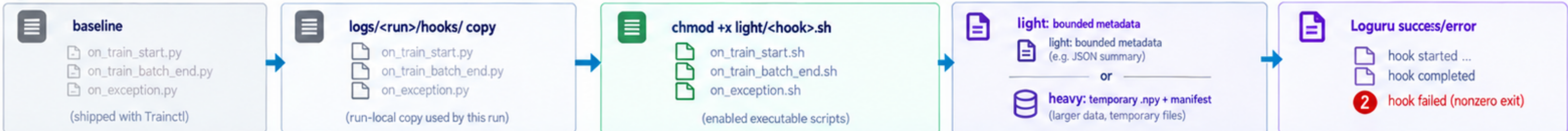


REST and FUSE read published state or enqueue commands; the training thread touches live Trainer/model state.

From symptom to evidence to a deliberate next step



lifecycle hooks | run synchronously as part of the training lifecycle (supporting, not the main flow)



The run keeps training semantics; every intervention is explicit, bounded, and visible.

Trainctl controls the live run. tboardfs makes its telemetry browsable.

Trainctl — live inspection and control



observe live state

metrics • ranks • learning rates
pipeline timing • system state



hold at safe points

cooperative pause • inspect safely
then continue



capture evidence

stacks • graphs • tensors
CUDA memory • profiles



apply targeted controls

set learning rate • reset momentum
set declared hparam • finite-value check

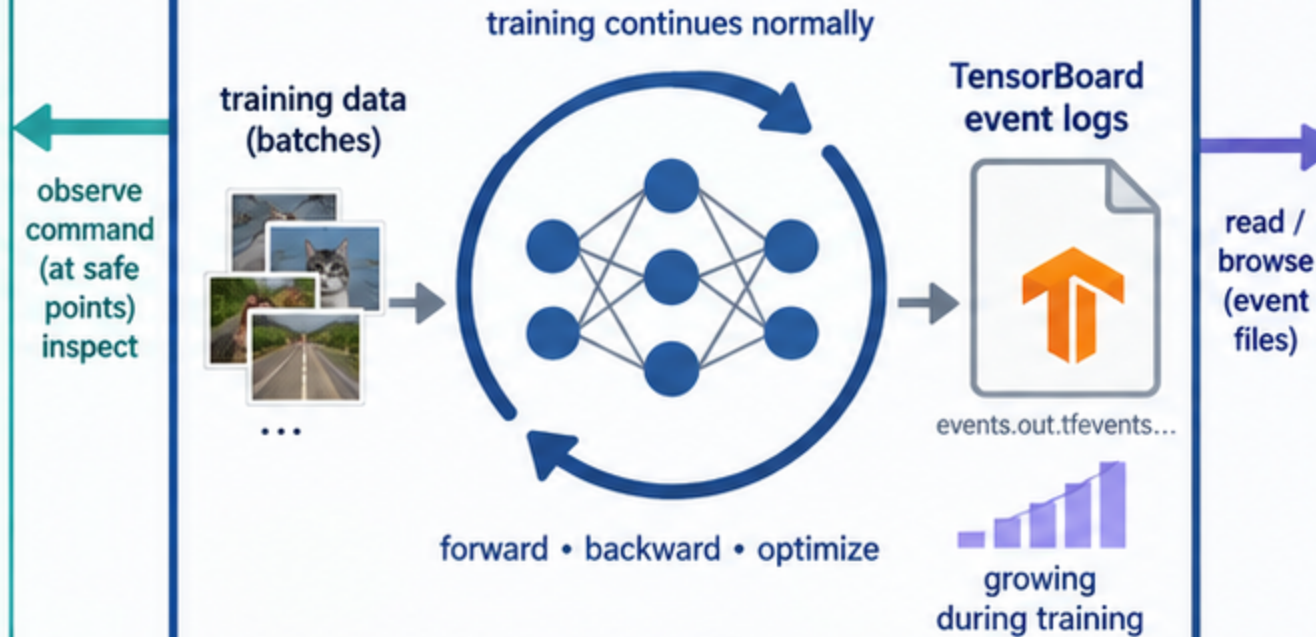


lifecycle hooks

epoch end • training end • exception

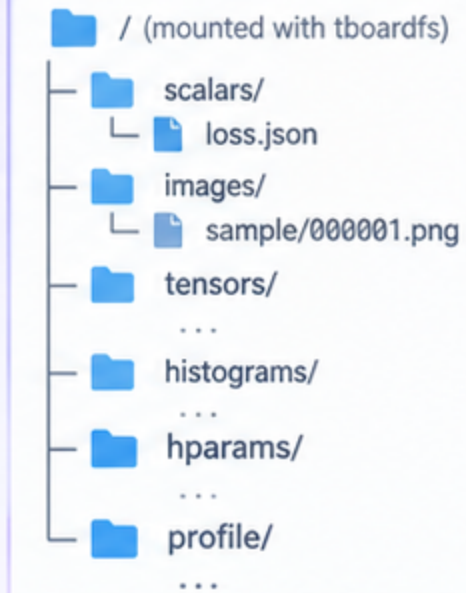
Lightning training run

ordinary Lightning run + TrainctlMixin



tboardfs — TensorBoard logs as ordinary files

FUSE filesystem mode (mount a log directory)



scalars, images, tensors, histograms,
distributions, text, meshes, PR curves,
hparams, graphs, plugin data ...

command mode (inspect one event file)

```
$ tboardfs list events.out...  
$ tboardfs get events.out...  
/scalars/loss.json  
$ tboardfs copy-all  
events.out... ./dump/
```

list • get • copy-all

same virtual paths across
filesystem and command modes

operator / agent

```
$ ls /scalars/  
loss.json acc.json ...  
$ cat /scalars/loss.json  
{ "step": 1000, "value": 0.123 }  
$ open /images/sample/000001.png
```



Trainctl answers:

what is the live process doing, and should I intervene?

one agent can use both

tboardfs answers:

what has the run logged, and what can I inspect as files?

Example workflow
(for humans or agents)

notice metric
regression

browse scalars/images
with tboardfs

hold with
Trainctl

capture deeper
evidence

continue

Live control + browsable telemetry = a much richer debugging surface for humans and agents.